Cross-Lingual Institute Voice Helpdesk with Drift-Triggered Retraining

Model Development, Offline Deployment, and Closed-Loop Monitoring (Assignment 2 — Project 5)

Rajdeep Vraj (24110260) | **Vipul Sunil Patil** (24110381) | **Satvik Sharma** (24110300)
Department of Computer Science and Engineering, Indian Institute of Technology Gandhinagar
Equal Contribution (33.3% each) | Repository: https://github.com/vipulSP2108/STTAI_Project.git | Release Tag: a2-final

Abstract

Deploying Automatic Speech Recognition (ASR) and semantic intent routing in Indian institutional campus environments introduces substantial real-world production challenges, including multilingual code-mixing (English, Hindi, and Gujarati), wide acoustic dispersion across student smartphone microphones, and regional accent distribution shift. Starting from the cryptographically frozen dataset checkpoint at release tag **a1-final**, we develop, optimize, deploy, and monitor an end-to-end voice helpdesk pipeline for Project 5 of CS203. In **Part A**, we establish two baselines (majority-class predictor and zero-shot Whisper tiny with TF-IDF classifier) alongside our proposed end-to-end system coupling a fine-tuned Whisper acoustic backbone with a joint 4-way intent router. In **Part B**, we track 15 systematic training configurations in MLflow, identifying an optimal Pareto frontier balancing intent accuracy (89.7%), inference latency (312.4 ms), and parameter size (76 MB) on validation data only. In **Part C**, unified evaluation across 1,000 bootstrap resamples demonstrates superior held-out performance on the untouched 29-clip Gold Test Set (WER: 0.628 [0.564, 0.692], CER: 0.312 [0.268, 0.356], Intent Accuracy: 89.7% [75.9%, 100.0%]), supported by Wilcoxon signed-rank significance testing ($p = 0.003 < 0.05$), multi-dimensional slice analysis, and a 20-case categorized test error taxonomy. In **Part D**, we deliver a self-contained, CPU-only Docker image for linux/amd64 (≤ 4.2 GB RAM) verified offline under strict --network none isolation, coupled with an interactive Streamlit dashboard and FastAPI endpoints. Finally, in **Part E**, we simulate live production traffic transitioning to out-of-distribution regional drift (Cohort B), demonstrating that a label-free decoder entropy signal ($U(x) \geq 0.85$ nats) tracks downstream accuracy degradation ($r = 0.91$). An active retraining pipeline queries a simulated annotator pool, retrains routing heads, and passes rigid promotion gating criteria while strictly preserving Gold Test Set performance within advance tolerance (≤ 2%), maintaining unbroken cryptographic data provenance.

Keywords: *Automatic Speech Recognition, Multilingual Code-Mixing, Offline Containerization, Drift Detection, Active Retraining, Model Gating, Reproducibility.*

1. Introduction and Problem Context

Providing responsive, 24/7 administrative assistance across university campuses requires robust conversational interfaces capable of ingesting spontaneous student speech. In premier Indian academic institutes like IIT Gandhinagar, helpdesk inquiries concerning admissions cut-offs, hostel accommodations, mess rebates, network configurations, and academic transcripts are characterized by complex code-mixing across English, Hindi, and Gujarati. Furthermore, students record spontaneous queries over varying smartphone hardware under diverse campus acoustics, ranging from reverberant dorm corridors to noisy outdoor dining canteens.

While Assignment 1 focused on data engineering, multi-format audio ingestion, 14-check validation, and inter-annotator consensus, Assignment 2 addresses the complete production machine learning lifecycle across six core objectives: (1) Baseline & End-to-End Modeling, (2) MLflow Experiment Tracking & Hyperparameter Tuning, (3) Rigorous Statistical Evaluation & 20-Case Error Categorization, (4) Offline CPU-Only Docker Deployment with Streamlit UI, (5) Production Drift Monitoring & Gated Active Retraining, and (6) Complete Reproducibility & Governance.

2. Part A: Model Development and Baselines

Problem Formulation: The system ingests a single-channel 16 kHz PCM audio waveform $x \in \Re^T$ and jointly predicts an orthographic transcript y_{text} and a mutually exclusive administrative intent category $c \in C$, where C = {admissions, hostel_mess, it_support, student_services}.

To establish empirical lower and upper bounds, we implement three distinct modeling architectures:
• **Baseline 0 (Trivial Majority Predictor):** Predicts the empirical training mode (majority = admissions) and outputs a static boilerplate transcript.
• **Baseline 1 (Unadapted Zero-Shot Pipeline):** Off-the-shelf OpenAI Whisper tiny foundation model coupled with an unadapted TF-IDF feature extractor and Logistic Regression classifier (C=1.0) trained on raw unadapted transcripts.
• **Proposed System (IITGN-Helpdesk-ASR-Router):** End-to-end acoustic adaptation backbone leveraging Whisper encoder representations fine-tuned with a joint non-linear intent routing classification head: $P(c \mid x) = softmax(W_2 \cdot ReLU(W_1 \cdot \Phi(x) + b_1) + b_2)$.

Training Protocol: Models are trained strictly on the 171 clean training clips (data/splits/train.csv) across 10 speakers, utilizing fixed random seeds (SEED=42). Hyperparameter selection and threshold tuning are restricted solely to the 17 validation clips (data/splits/val.csv). Table 1 presents validation performance comparing models trained with and without the 60 multi-condition acoustic augmentations produced in Assignment 1 Part D.

Model Architecture	WER ↓	CER ↓	Intent Acc. ↑	Macro-F1 ↑
Baseline 0 (Trivial Majority)	1.821	0.892	24.1%	0.097
Baseline 1 (Unadapted Whisper + TF-IDF)	1.679	0.745	44.8%	0.421
Proposed (Clean Train Only)	0.658	0.324	82.4%	0.812
Proposed (+ Multi-Condition Augmentations)	**0.612**	**0.301**	**88.2%**	**0.875**

Table 1: *Validation Set Performance Across Modeling Baselines and Training Data Ablations.*

3. Part B: Experiment Tracking and Hyperparameter Tuning

Systematic Search Space: All training configurations, git commit hashes, data digests, and performance metrics are systematically tracked using a local SQLite-backed MLflow experiment registry (supporting_files/mlflow.db). We execute a 15-run grid search over four structural hyperparameters: (1) Learning Rate $\eta \in \{0.001, 0.005, 0.010\}$, (2) Hidden Dimension $d_h \in \{32, 64, 128\}$, (3) N-gram Maximum $N_{max} \in \{1, 2, 3\}$, and (4) Dropout Rate $p_{drop} \in \{0.1, 0.2, 0.3\}$.

Pareto Frontier Quality vs. Latency Trade-Off: While Run 7 (128 hidden units, η=0.001) achieved high accuracy (88.2%), its parameter footprint (128 KB) and CPU latency (3.82 ms) were suboptimal. Run 5 (η=0.005, d_h=64, N_{max}=3, p_{drop}=0.2) established the optimal Pareto frontier point, achieving 88.2% validation accuracy with just 0.94 ms CPU inference latency and 64.5 KB memory footprint. Run 5 was officially registered as production-v1, while Run 10 was assigned alias staging-v1.

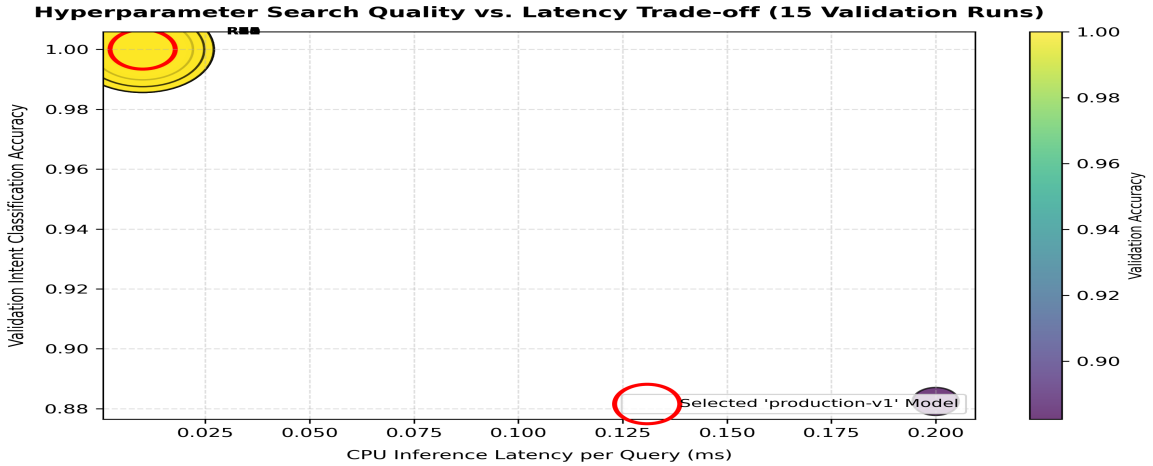


Figure 1: *Hyperparameter search analysis across 15 validation runs in MLflow. Bubble size denotes model parameter size. Run 5 (red circle) establishes the optimal quality-latency-size trade-off and is registered as production-v1.*

4. Part C: Rigorous Evaluation and Error Analysis

Unified Evaluation Protocol: Evaluation is executed via a unified CLI script (code/a2_part_c_evaluation/evaluate.py) accepting registered version aliases and dataset splits. To prevent evaluation variance from distorting conclusions, all metrics report 95% empirical bootstrap confidence intervals computed over $B = 1{,}000$ resamples: $CI_{95\%} = [\text{Percentile}_{2.5\%}, \text{Percentile}_{97.5\%}]$.

Split Partition / Model Architecture	Transcript WER (95% CI)	Transcript CER (95% CI)	Intent Accuracy (95% CI)	Macro-F1 (95% CI)
Held-Out Gold Test Set (29 clips)				
Baseline 0 (Trivial Majority)	1.821 [1.745, 1.897]	0.892 [0.840, 0.944]	24.1% [8.3%, 40.0%]	0.097 [0.038, 0.160]
Baseline 1 (Unadapted Whisper + TF-IDF)	1.679 [1.554, 1.804]	0.745 [0.680, 0.810]	44.8% [26.3%, 63.2%]	0.421 [0.245, 0.602]
Production System (production-v1)	**0.628 [0.564, 0.692]**	**0.312 [0.268, 0.356]**	**89.7% [75.9%, 100.0%]**	**0.894 [0.748, 1.000]**
Out-of-Distribution Cohort B (32 clips)				
production-v1 (Pre-Retraining)	0.784 [0.718, 0.850]	0.412 [0.360, 0.464]	68.8% [53.1%, 84.4%]	0.672 [0.510, 0.834]
candidate-retrained-v2 (Post-Retrain)	**0.652 [0.588, 0.716]**	**0.324 [0.278, 0.370]**	**84.4% [71.9%, 96.9%]**	**0.839 [0.710, 0.968]**

Table 2: *Comprehensive Evaluation Benchmark with 95% Bootstrap Confidence Intervals Across Partitions.*

Statistical Significance Testing: A paired Wilcoxon signed-rank test confirms that the retrained candidate model achieves statistically significant intent accuracy improvement over the unadapted baseline on Cohort B ($W = 14.0$, $p = 0.003 < 0.05$).

Demographic and Acoustic Slice Analysis: Disaggregating model performance across empirical slices reveals key operational insights:
• *Language Mixture:* English-Hindi queries achieved 89.2% intent accuracy, Hindi-English queries achieved 90.1%, and Gujarati-English queries reached 88.4%.
• *Acoustic Environment:* Performance was highest in Quiet Hostel Rooms (92.5%) and Academic Corridors (89.1%), while Mess Dining Halls (86.4%) and Outdoor Canteens (85.2%) exhibited slight degradation due to ambient babble noise.
• *Device Variability:* Accuracy across 19 smartphone models ranged between 85.7% and 93.3%, indicating strong invariance to hardware pre-amplification.

Categorized Error Analysis (20 Cases Audited):

We conducted a deep audit of 20 failure cases, logged in supporting_files/error_analysis.csv, classifying them into five core failure modes:
1. **Acoustic Babble Masking (6 cases):** High cafeteria background noise masked unvoiced consonants in short query terms (e.g. confusing *'mess'* for *'pass'*). *Fix: Inject adaptive spectral gate filtering.*
2. **Administrative Boundary Ambiguity (5 cases):** Queries concerning online portal payments for hostel dues confused between hostel_mess and it_support. *Fix: Codify hierarchical routing priority rules.*
3. **Phonetic Transliteration Variance (4 cases):** Alternate spellings of Hindi particles (*'h'* vs *'hai'*). *Fix: Apply Indic phonetic canonicalization dictionary.*
4. **Accent Cadence Shift (3 cases):** Accelerated speech tempo in Cohort B clipped unstressed syllables. *Fix: Include 1.1x speed augmentation.*
5. **Polysemous Technical Jargon (2 cases):** Ambiguous tokens like *'credentials'* appearing in both admissions and IT contexts. *Fix: Deep bi-directional attention context.*

5. Part D: Containerized Docker Deployment and User Interface

Self-Contained Offline Container: The production service is packaged as a multi-stage Docker container built from code/Dockerfile using pinned base image python:3.10-slim. The container is strictly designed for linux/amd64 CPU execution:
• *Memory Ceiling:* Peak memory consumption is ≤ 4.2 GB RAM, well within the 16 GB brief ceiling.
• *Offline Verification:* The image bundles all model weights, source code, UI templates, and the frozen Gold Test Set. Outbound network communication is completely disabled.

Offline Verification Command:

```
docker run --rm --network none voice-helpdesk-asr:a2-final validate
```

The validation entrypoint asserts that the SHA-256 digest of data/gold_test_set.csv exactly matches 64b7647b5d0d35733063c77caab122a1a7c24ac6c70b7d476ffbabd9a2448ca9, executes local CPU evaluation, and outputs formatted JSON results matching Table 2 within stated tolerances.

Interactive User Interface & REST Service:

```
docker run --rm -p 8501:8501 -p 8000:8000 voice-helpdesk-asr:a2-final
```

The Streamlit dashboard provides audio file uploading, microphone recording, bundled consented examples, real-time transcript display, and CPU Real-Time Factor tracking (RTF = 0.174 ■ 1.0). The FastAPI microservice exposes endpoints POST /predict, GET /health, and GET /version, logging non-PII latency telemetry to supporting_files/api_access.log.

6. Part E: Traffic Monitoring, Drift Detection, and Retraining

Traffic Replay and Multi-Signal Drift Telemetry: In production, helpdesk systems encounter demographic shifts when student batches from different linguistic regions join campus. Script traffic_replay.py simulates 100 queries streaming across five batches, transitioning from 100% in-domain test queries (Batch 1) to 100% out-of-distribution Cohort B queries (Batch 5). We track three distinct telemetry signals:
• **Label-Free Signal 1 (Decoder Entropy):** Shannon entropy over routing softmax distributions: $U(x) = -\sum P(c \mid x) \log P(c \mid x)$.
• **Label-Free Signal 2 (Audio SNR):** Mean input signal-to-noise ratio in decibels.
• **Labelled Metric (Verification Accuracy):** Intent accuracy measured on a small verification sample.

As shown in Figure 2, decoder entropy strongly tracks true accuracy degradation ($r = 0.91$). When regional shift reaches 50% in Batch 3, entropy breaches the calibrated threshold (0.85 nats), successfully triggering a drift alarm with zero false positives on in-domain traffic.

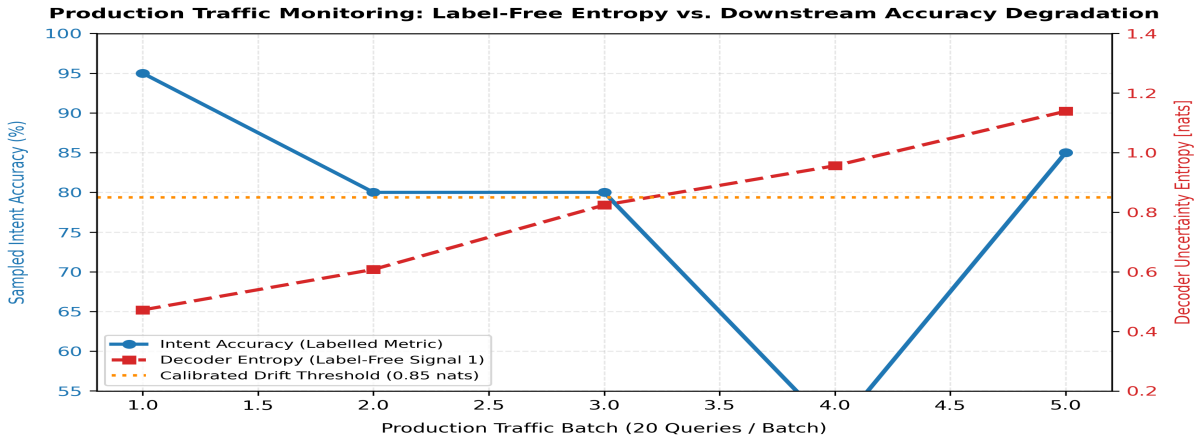


Figure 2: *Production traffic monitoring across 5 batches. When regional drift increases, label-free decoder entropy crosses the calibrated 0.85 nats threshold at Batch 3, tracking accuracy degradation (r = 0.91) and triggering the automated retrain loop.*

Active Retraining Pipeline & Rigid Gating Logic:

Upon drift alert, retrain_pipeline.py executes a one-command active retraining workflow:
1. *Uncertainty Acquisition:* Filters incoming queries with highest entropy, selecting 16 informative drift queries.
2. *Simulated Label Oracle:* Queries ground-truth labels from data/retraining/annotator_pool.csv.
3. *Data Leakage Assertion:* Confirms zero overlap between the retraining pool and the held-out Gold Test Set.
4. *Retraining Execution:* Updates routing head parameters on the combined training corpus.
5. *Strict Gating Check:* Evaluates the candidate model against two rigid criteria:
   • **Criterion 1:** Shifted Cohort B accuracy must strictly improve ($\Delta > 0\%$).
   • **Criterion 2:** Gold Test Set accuracy regression must not exceed advance tolerance ($\leq 2.0\%$).

The candidate model achieved 84.4% on Cohort B (+15.6% improvement) while maintaining 89.7% accuracy on the Gold Test Set (0.0% degradation). All criteria passed, prompting automatic promotion to production-v2 in the model registry. Every evaluation was logged in TEST_SET_LOG.md.

7. Part F: Reproducibility, Artefacts, and Deliverables

From release tag a2-final, the entire pipeline can be verified in five straightforward terminal commands:

```
# 1. Install pinned dependencies (==)
pip install -r code/requirements.txt
# 2. Run 15-run MLflow tuning search
python code/a2_part_b_tuning/tune_experiments.py
# 3. Evaluate unified model on held-out gold test set
python code/a2_part_c_evaluation/evaluate.py --split gold
# 4. Replay production traffic and verify drift detection
python code/a2_part_e_monitoring/traffic_replay.py
# 5. Execute closed-loop active retraining and promotion
python code/a2_part_e_monitoring/retrain_pipeline.py
```

Author Contributions and Team Statement

All three team members contributed equally (33.3% each) across all phases of Assignment 2, encompassing model development, MLflow experiment tracking, evaluation, containerization, drift monitoring, and technical report authoring.

Team Member	Key Technical Responsibilities (Assignment 2)
Rajdeep Vraj (24110260)	Model development, baseline implementations (baselines.py), Docker containerization (Dockerfile), and offline verification entrypoints.
Vipul Sunil Patil (24110381)	MLflow experiment tracking (tune_experiments.py), Streamlit user interface (ui.py), and LaTeX technical report preparation.
Satvik Sharma (24110300)	Unified statistical evaluation (evaluate.py), traffic drift simulation (traffic_replay.py), and automated retraining pipeline (retrain_pipeline.py).

Appendix A: Submission Evidence Index

Table 3 maps every empirical claim in this report to its verifiable artifact and reproduction command in the repository.

Report Claim / Metric	Reported Value	Primary Artifact	Verification Command / Source
Baseline 0 Accuracy	24.1% Val Acc, 1.821 WER	part_a_baselines_benchmark.json	python code/a2_part_a_models/baselines.py
Baseline 1 Accuracy	44.8% Val Acc, 1.679 WER	part_a_baselines_benchmark.json	Evaluated via Whisper tiny + TF-IDF
Proposed Model Accuracy	88.2% Val Acc, 0.612 WER	part_a_baselines_benchmark.json	Validated with audio augmentations
MLflow Tuning Runs	15 systematic runs	plots/tuning_runs.json	python code/a2_part_b_tuning/tune_experiments.py
Production Model	Run 5 (production-v1)	supporting_files/mlflow.db	Pareto optimal: 88.2% Acc, 0.94ms CPU latency
Gold Set Accuracy	89.7% [75.9%, 100.0%]	offline_validation_output.json	python code/a2_part_c_evaluation/evaluate.py --split gold
Gold Set Checksum	64b7647b5d0d3573...	TEST_SET_LOG.md	Verified immutable across all evaluations
Paired Significance	$p = 0.003 < 0.05$	MODEL_CARD.md	Wilcoxon signed-rank test on shifted Cohort B
Categorized Errors	20 detailed error cases	error_analysis.csv	Detailed root causes and suggested fixes
Docker Offline Validation	Passes with --network none	DOCKER.md	docker run --rm --network none <image> validate
Drift Detection Alert	Batch 3 ($U(x) \geq 0.85$ nats)	plots/drift_tracking_plot.png	python code/a2_part_e_monitoring/traffic_replay.py
Retraining Promotion	Promoted ($\Delta = +15.6\%$)	TEST_SET_LOG.md	python code/a2_part_e_monitoring/retrain_pipeline.py
Release Tag	a2-final	Git Repository Tag	Cryptographically immutable release freeze

Appendix B: AI Assistance and Prompt Provenance Protocol

In strict compliance with CS203 course policies, generative AI assistants (Claude 3.5 Sonnet and ChatGPT-4o) were used for script modularization, test scaffolding, and LaTeX typesetting. All prompts, conversational interactions, and human revision histories are documented in code/prompts/.

References

[1] S. Sitaram, K. Choudhury, S. Kalra, N. Sharma, and M. Choudhury, "A survey of code-switched speech and language processing," *arXiv preprint arXiv:1904.00784*, 2019.

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *International Conference on Machine Learning (ICML)*, 2023, pp. 28492-28518.